

Sparse Autoencoders for GPT-2: A Hands-On Tutorial

This tutorial walks through what Sparse Autoencoders (SAEs) are, why we use them on language models, and how our implementation works — with code references so you can read this alongside `sae.py`, `train_sae.py`, and `eval_sae.py`.

1. What is a Sparse Autoencoder?

An **autoencoder** is a neural net that learns to compress data and then reconstruct it:

```

input x  --> [Encoder] --> hidden h --> [Decoder] --> reconstructed
x_hat
(768,)      (768 -> 6144)  (6144,)      (6144 -> 768)  (768,)

```

The twist with a **sparse** autoencoder: we make the hidden layer much *wider* than the input (e.g., 768 -> 6144), but force most hidden units to be zero. This means:

- The network has **more features than input dimensions** (overcomplete)
- But only a handful activate for any given input (~30 out of 6144)
- Each feature learns to represent one specific, interpretable concept

Think of it like a dictionary: you have 6144 "words" (features), but any given sentence (activation vector) only uses ~30 of them.

2. Why Hook It to a Language Model?

Neural networks like GPT-2 represent concepts as **directions in activation space**. But here's the problem: a 768-dimensional space needs to represent far more than 768 concepts. The model solves this through **superposition** — it overlaps multiple concepts in the same dimensions.

This means individual neurons don't correspond to clean concepts. Neuron #42 in layer 6 might partially encode "is a proper noun," "appears after a comma," and "is related to science" all at once.

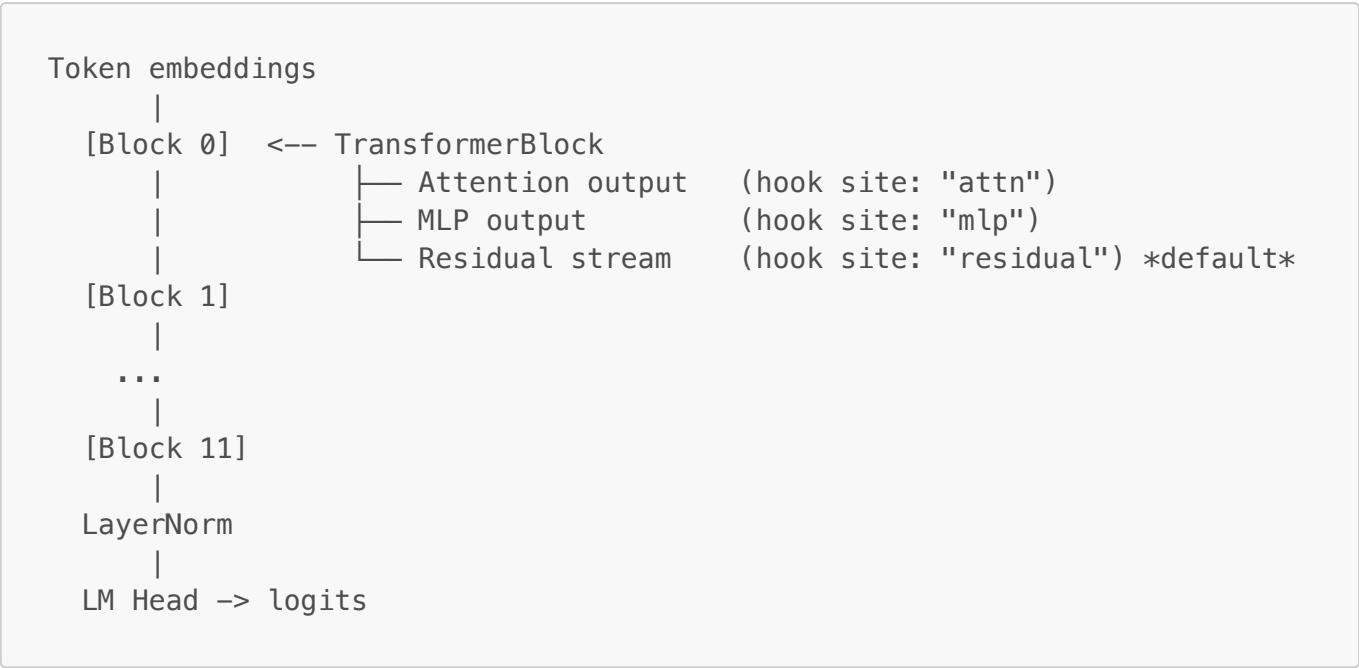
SAEs disentangle this. By projecting into a much wider sparse space, each SAE feature can isolate one concept:

- Feature #1023 might fire on Python code keywords
- Feature #4501 might fire on names of countries
- Feature #892 might fire on tokens following a question mark

This gives us a **microscope into what GPT-2 is thinking** at any given layer.

Where do we hook?

We use PyTorch's `register_forward_hook` to intercept activations at a specific point in the transformer. The main options:



Residual stream (the output of a full transformer block) is the most common choice. It's the "main highway" of information flow — everything the model knows at that layer is encoded here.

In our code (`train_sae.py`), hooking looks like this:

```
# That's it - 3 lines to intercept activations from any layer
activations = {}
def hook_fn(module, input, output):
    activations["value"] = output.detach()

handle = model.blocks[layer].register_forward_hook(hook_fn)
```

After any `model(tokens)` call, `activations["value"]` holds the residual stream at that layer, shape `(batch, seq_len, 768)`.

3. The Math

Our SAE has four learnable parameters (see `sae.py`):

Parameter	Shape	Role
<code>W_enc</code>	<code>(d_sae, d_model) = (6144, 768)</code>	Encoder weights
<code>b_enc</code>	<code>(d_sae,) = (6144,)</code>	Encoder bias
<code>W_dec</code>	<code>(d_model, d_sae) = (768, 6144)</code>	Decoder weights
<code>b_dec</code>	<code>(d_model,) = (768,)</code>	Decoder bias

Forward pass (4 lines)

```

h      = ReLU(W_enc @ (x - b_dec) + b_enc)  # (1) encode
x_hat  = W_dec @ h + b_dec                  # (2) decode
mse    = (x - x_hat).pow(2).mean()          # (3) reconstruction loss
l1     = h.abs().mean()                     # (4) sparsity loss
loss   = mse + l1_coeff * l1                # total loss

```

Let's unpack each line:

(1) Encode: $h = \text{ReLU}(W_{\text{enc}} @ (x - b_{\text{dec}}) + b_{\text{enc}})$

- First we **center** the input by subtracting b_{dec} . Why? The decoder bias will learn the mean activation vector. By subtracting it first, the encoder only needs to represent the *residual* — how this activation differs from average. This makes learning easier.
- Then we project into the wide sparse space with W_{enc} and add a bias b_{enc} .
- **ReLU** zeroes out negative values, enforcing sparsity. Most features will output zero.

(2) Decode: $x_{\text{hat}} = W_{\text{dec}} @ h + b_{\text{dec}}$

- Project back to the original dimension. Each column of W_{dec} is a **feature direction** — it represents what that feature "means" in activation space.
- Add back b_{dec} (the mean) to reconstruct the full activation.

(3) Reconstruction loss: $\text{mse} = (x - x_{\text{hat}}).pow(2).mean()$

- We want the reconstruction to be faithful. MSE measures how well the SAE can compress and decompress without losing information.

(4) Sparsity loss: $l1 = h.abs().mean()$

- L1 penalty encourages the hidden activations to be sparse (lots of zeros).
- $l1_coeff$ controls the tradeoff: too low and features aren't sparse enough, too high and reconstruction suffers.

Why unit-norm decoder columns?

We normalize each column of W_{dec} to have L2 norm = 1. Without this, the model could "cheat": scale up W_{dec} columns and shrink h to compensate — the reconstruction $W_{\text{dec}} @ h$ stays the same, but h is smaller so L1 loss looks low without any actual sparsity. Unit-norm columns prevent this — the only way to reduce L1 is genuine sparsity.

In code (`train_sae.py`):

```

# One line, every 100 steps
sae.W_dec.data /= sae.W_dec.data.norm(dim=0, keepdim=True)

```

4. Training Intuition

What does the loss curve look like?

You'll typically see:

- **MSE** starts high and drops quickly in the first few thousand steps, then slowly improves
- **L1** stays relatively stable after an initial adjustment period
- **Total loss** follows MSE since it dominates

Key metrics to watch

L0 (average active features):

```
L0 = (h > 0).float().mean() # fraction of features that are non-zero
```

- Multiply by `d_sae` to get the average count. For a well-trained SAE, this might be 20-50 out of 6144.
- If L0 is too high (>100), increase `l1_coeff`. If too low (<10), decrease it.

Explained variance:

```
explained_var = 1 - (x - x_hat).var() / x.var()
```

- How much of the activation's variance the SAE captures. Aim for >0.90.
- If this is low, the SAE is losing too much information (maybe `l1_coeff` is too high).

Dead features:

- Features that never activate across many batches. They've been "wasted."
- Some dead features are normal (10-20%), but if >50% are dead, something is wrong.
- Common fixes: lower `l1_coeff`, increase learning rate, or add dead neuron resampling (a more advanced technique we skip for now).

Typical hyperparameter starting points

Parameter	Value	Notes
<code>d_sae</code>	768 * 8 = 6144	8x expansion, good default
<code>l1_coeff</code>	5e-3	Tune based on L0
<code>lr</code>	3e-4	Adam, standard
<code>hook_layer</code>	6	Middle layer, balanced features
<code>total_tokens</code>	10M-100M	More is better, but diminishing returns

5. Reading the Results

After training, `eval_sae.py` lets you inspect what the SAE learned.

Top activating tokens

For a given feature (say #42), we find the tokens where that feature activates most strongly:

```
Feature 42 – top activating tokens:
  "Python"      activation: 8.3   context: "...written in Python and
uses..."
  "JavaScript"  activation: 7.1   context: "...frameworks like JavaScript
or..."
  "Java"        activation: 6.8   context: "...programming in Java for..."
  "C++"         activation: 5.9   context: "...compiled from C++
source..."
```

If the top tokens cluster around a theme, the feature has learned something interpretable. This feature seems to detect programming language names.

What to look for

- **Monosemantic features:** Activate on one clear concept (best case)
- **Polysemantic features:** Activate on 2-3 related concepts (common, still useful)
- **Dead features:** Never activate (wasted capacity)
- **Noise features:** Activate on seemingly random tokens (undertrained or too many features)

A well-trained SAE typically has:

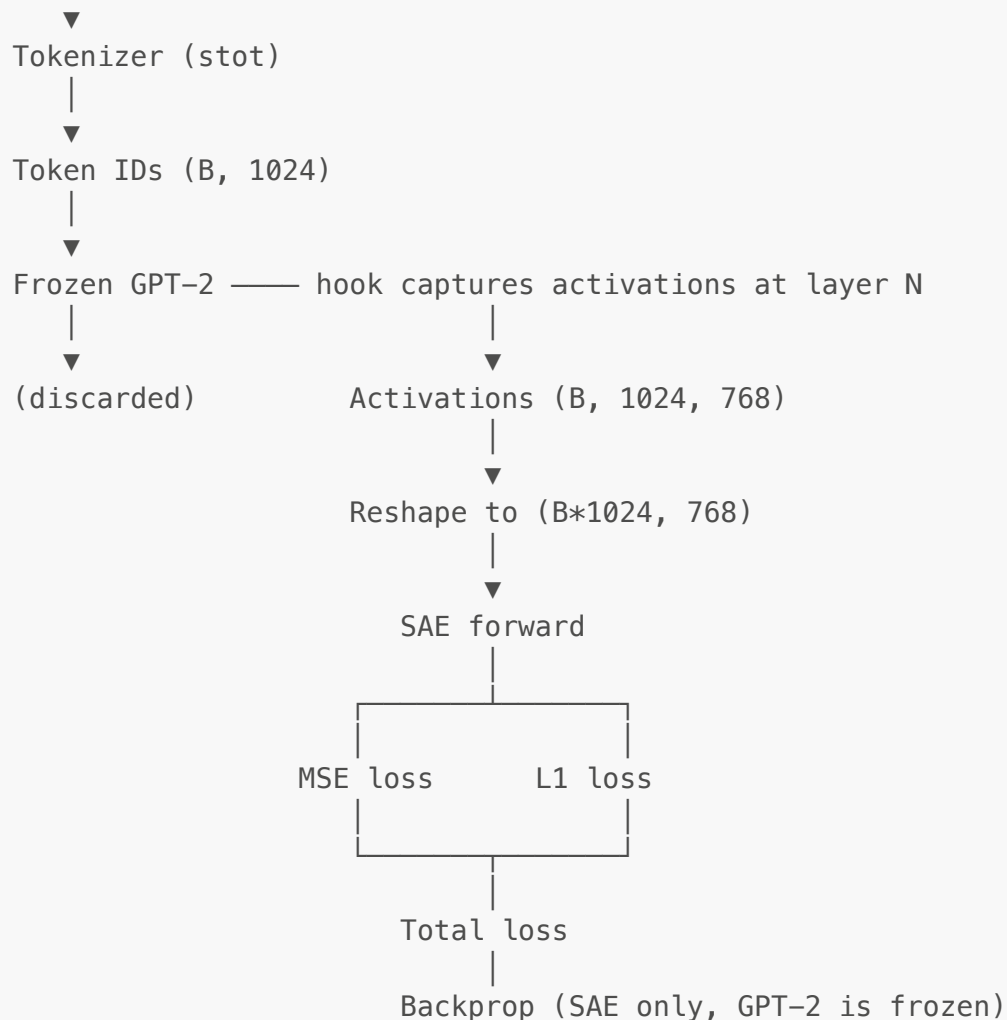
- 30-60% clearly interpretable features
- 20-40% somewhat interpretable
- 10-20% dead or noisy

6. Code Map

File	What it does	Key function
<code>sae.py</code>	SAE model definition	<code>SparseAutoencoder.forward()</code> — the 4-line forward pass
<code>train_sae.py</code>	Trains the SAE on GPT-2 activations	Main training loop with hook attachment
<code>eval_sae.py</code>	Analyzes trained SAE features	<code>top_activating_tokens()</code> , <code>reconstruction_stats()</code>

How data flows during training

```
FineWeb-Edu text
|
```



Next Steps (after you've trained a basic SAE)

Things you can explore once the basics work:

1. **Different layers:** Train SAEs on layers 0, 3, 6, 9, 11 and compare what features emerge
2. **Different hook sites:** Try MLP output (`--hook_site mlp`) vs residual stream
3. **TopK activation:** Replace ReLU + L1 with keeping only the top-k activations (no L1 needed)
4. **Dead neuron resampling:** Recycle dead features by reinitializing them toward high-error inputs
5. **Larger expansion:** Try 16x or 32x for more fine-grained features
6. **Substitution test:** Replace original activations with SAE reconstructions and measure how much GPT-2's outputs change (KL divergence)